

Progetto B1: Service Registry Distribuito

Giulia Boccuccia

Ingegneria Informatica

Università degli Studi di Roma Tor Vergata

Roma, Italia

giuliaboccuccia@gmail.com

Abstract—Il progetto realizza in Go un Service Registry distribuito, decentralizzato, nel quale ogni nodo mantiene una replica completa dello stato e gestisce autonomamente le richieste di registrazione, aggiornamento, discovery e cancellazione delle istanze. La propagazione degli aggiornamenti combinano notifiche push e sincronizzazione periodica anti-entropia. Versioni logiche, risoluzione deterministica dei conflitti e tombstone consentono la convergenza delle repliche al ripristino della comunicazione.

I. INTRODUZIONE

Il progetto realizza un Service Registry interamente in GO, il cui obiettivo è consentire la registrazione, l'aggiornamento, la discovery e la rimozione dinamica dei servizi senza dipendere da un server centrale. L'indisponibilità di uno dei nodi non impedisce agli altri nodi di continuare a effettuare la registrazione e il discovery.

Lo stato del registry viene replicato ed è possibile che fra i nodi sia divergente. Il sistema adotta un modello di consistenza finale: in assenza di nuovi aggiornamenti, a condizione che la comunicazione tra i nodi sia disponibile e consenta la propagazione dei record, le repliche convergono deterministicamente verso lo stesso stato. Per raggiungere questo risultato sono utilizzati principalmente cinque meccanismi:

- 1) replica completa del registry su ciascun nodo;
- 2) versioning logico dei record;
- 3) risoluzione deterministica dei conflitti;
- 4) propagazione immediata tramite gossiping;
- 5) sincronizzazione periodica anti-entropia.

II. ARCHITETTURA DEL SISTEMA

L'architettura adottata è costituita da tre nodi registry principali, configurati tramite **Docker Compose**. Ogni nodo espone autonomamente le API, mantiene una copia locale del registry, comunica con gli altri peer e salva periodicamente il proprio stato su uno storage persistente. I tre registry sono quindi peer equivalenti: l'indisponibilità temporanea di uno di essi non impedisce agli altri di continuare a servire le richieste.

A. Replica completa dello stato

Ogni nodo mantiene una replica completa del registry. Una richiesta di discovery può essere soddisfatta utilizzando esclusivamente lo stato locale, senza dover interagire con gli altri nodi. Questo riduce la dipendenza dalla rete durante le operazioni ordinarie e rende ciascun registry autonomo nella risposta.

B. Modello di consistenza

Il sistema implementa un modello di **eventual consistency**. Dopo un aggiornamento è possibile che per un intervallo di tempo, dipendente dalla comunicazione e dai tempi di sincronizzazione, nodi differenti possiedano versioni differenti dello stato. Il requisito fondamentale è che in assenza di nuovi aggiornamenti e una volta ripristinata la comunicazione, tutti i nodi convergano. Questa scelta privilegia la disponibilità: un nodo può continuare a registrare, cancellare e restituire le istanze anche quando un peer è temporaneamente irraggiungibile.

III. STRATEGIA DI REPLICAZIONE

Quando eseguiamo una modifica, attraverso le API pubbliche, l'handler aggiorna il registry e notifica il componente **Gossiper**. Il gossiping ha lo scopo di ottenere una bassa latenza di propagazione, comunicando rapidamente lo stato agli altri peer.

Richiesta HTTP → Handler → Registry → Snapshot aggiornato → Gossiper → Peer.

Il gossiping viene gestito in background e non determina il completamento di una richiesta da parte di un client. In questo modo la replicazione è asincrona rispetto all'operazione principale.

Il gossiping da solo non garantisce il recupero degli aggiornamenti persi durante una interruzione di servizio. Per questo è stato introdotto il componente **Synchronizer**, che esegue periodicamente una sincronizzazione dello stato. All'interno del progetto l'intervallo di sincronizzazione è stato settato a SYNC_INTERVAL=5s.

Ogni nodo interroga periodicamente i peer e integra gli aggiornamenti mancanti. Il sistema combina quindi gossiping e anti-entropia per avere rapidità e garanzie di convergenza. La propagazione push invia lo snapshot completo ai peer configurati, che sono contattati in sequenza. Il componente Gossiper utilizza una coda limitata, nella quale una nuova notifica può sostituire quella in attesa. Il Synchronizer acquisisce gli snapshot completi dei peer tramite richieste GET e ne integra i record; esegue un tentativo iniziale all'avvio e successivamente si attiva con un intervallo configurato di 5s. La lista dei peer è fornita tramite configurazione.

Il Gossiper è un meccanismo event-driven che, dopo una modifica locale, tenta immediatamente di propagare lo stato aggiornato verso gli altri peer; il Synchronizer è invece un

meccanismo time-driven di anti-entropy che periodicamente interroga gli altri nodi e integra nel registry locale gli aggiornamenti eventualmente persi.

IV. RAPPRESENTAZIONE DELLO STATO

Sono presenti due strutture fondamentali `ServiceRecord` e `InstanceRecord`.

La distinzione fondamentale fra le due strutture è quella che: il **servizio** rappresenta un'entità logica identificata dal nome; mentre, ciascuna **istanza** rappresenta un endpoint concreto che offre quel servizio ed è identificata dalla coppia `(ServiceName, ID)`.

Listing 1. Implementazione del Service Record

```
1 type ServiceRecord struct {
2     Name      string
3     Status    Status
4     Version   uint64
5     OriginNode string
6     UpdatedAt time.Time}
```

Questa struttura rappresenta un servizio. Il servizio rappresenta una categoria, che può avere più istanze.

Il campo `version` consente di configurare aggiornamenti dello stesso record. Il campo `originNode` identifica il nodo che ha generato l'aggiornamento e risolve i conflitti quando due record hanno la stessa versione. Aggiornamenti concorrenti possono avere la stessa versione ma la concorrenza non implica necessariamente l'uguaglianza delle versioni.

Listing 2. Implementazione del Instance Record

```
1 type InstanceRecord struct {
2     ID          string
3     ServiceName string
4     Address     string
5     Port        int
6     Status      Status
7     Version     uint64
8     OriginNode  string
9     UpdatedAt   time.Time
10 }
```

Rappresenta una singola istanza di un servizio.

Il campo `status` di una istanza è separato dal campo del servizio in quanto è possibile che un'istanza possa essere eliminata, ma il servizio può rimanere attivo.

V. VERSIONING E RISOLUZIONE DEI CONFLITTI

All'interno del nostro sistema è possibile che due nodi possano modificare contemporaneamente lo stesso dato. Il contatore logico della versione è locale al nodo ed è condiviso dalle operazioni sui diversi record. La versione cresce quando viene generato un nuovo record aggiornato; una singola richiesta può quindi consumare più versioni, per esempio quando si crea sia il servizio che la sua prima istanza. Una cancellazione ripetuta sul record già eliminato non incrementa la versione. Per la risoluzione dei conflitti utilizziamo una regola deterministica basata sulla coppia `(Version, OriginNode)`.

- prevale il record con `version` maggiore;
- a parità di versione, prevale `OriginNode` maggiore secondo l'ordinamento lessicografico;

- se entrambi coincidono, il record remoto viene ignorato.

Tutti i nodi applicano questa regola, quindi partendo dallo stesso insieme di aggiornamenti convergono sulla stessa versione del record senza bisogno di un contatore centrale.

Il campo `UpdatedAt` descrive temporalmente l'aggiornamento, ma non è preso in considerazione per la risoluzione dei conflitti, in quanto vogliamo evitare la sincronizzazione dei clock fisici.

Ogni modifica locale, incluse le cancellazioni, incrementa la versione logica. Quando un nodo riceve una versione remota più alta fa avanzare anche il proprio contatore locale.

VI. GESTIONE DELLE CANCELLAZIONI MEDIANTE CANCELLAZIONE LOGICA

La cancellazione è rappresentata da un tombstone: il record viene marcato e mantenuto con stato `deleted` e una nuova versione. Questo impedisce a una copia attiva precedentemente di prevalere durante il merge. La cancellazione di un'istanza lascia attivo il servizio, mentre quella di un servizio marca come eliminato anche le sue istanze localmente attive. La discovery restituisce le istanze soltanto se il servizio e l'istanza risultano attivi. Una successiva registrazione può riattivare il servizio o l'istanza con una nuova versione.

VII. API HTTP

Il progetto utilizza il package standard `Go net/http`. Gli endpoint HTTP implementati sono:

Metodo	Endpoint	Funzione
GET	/health	Health check
PUT	/services/name/instances/{id}	Registrazione/aggiornamento
GET	/services/{name}	Discovery
DELETE	/services/{name}/instances/{id}	Cancellazione delle istanze
DELETE	/services/{name}	Cancellazione servizio e istanze
GET	/internal/state	Acquisizione snapshot interno
PUT	/internal/state	Merge snapshot remoto

Gli endpoint interni sono destinati alla replicazione e il codice li espone sullo stesso server HTTP delle API pubbliche.

VIII. CONCORRENZA

Il registry viene usato in contemporanea da richieste HTTP, gossip, anti-entropy e persistenza periodica. Quindi è necessario proteggere lo stato condiviso.

Lo stato condiviso del registry è protetto da un `sync.RWMutex`. Le operazioni di lettura, come discovery e snapshot, acquisiscono un lock condiviso; mentre gli aggiornamenti acquisiscono un lock esclusivo. Il merge di uno snapshot si applica ai record separatamente, quindi l'intero merge non costituisce un'unica operazione atomica.

Anche il sistema di reporting degli errori in background è progettato per essere concurrency-safe.

IX. PERSISTENZA

Ogni nodo può salvare lo stato del registry su un file JSON:
STATE_FILE=/data/registry-state.json
PERSIST_INTERVAL=1s

Ogni salvataggio viene effettuato in modo atomico dal componente `FileStore.Save`, invocato dal componente **Persister**. Il procedimento è il seguente:

- 1) lo snapshot viene scritto in un file temporaneo;
- 2) il contenuto viene sincronizzato su disco;
- 3) il file temporaneo sostituisce quello precedente tramite `rename`.

All'avvio il nodo carica lo stato persistito ed esegue il merge, ripristinando anche il contatore logico. Se il file non esiste, il nodo parte con uno stato vuoto.

Se il file esiste, ma il caricamento o la decodifica del JSON falliscono, il nodo interrompe l'avvio. Il decoder rifiuta anche campi sconosciuti e valori JSON aggiuntivi.

La persistenza è periodica e utilizza un volume distinto per ciascun nodo. Sono previsti salvataggi anche durante l'arresto; la risposta a una scrittura non attende tuttavia la persistenza su disco. Un arresto improvviso può quindi perdere gli aggiornamenti locali ancora non salvati, recuperabili dai peer soltanto se già replicati.

X. AWS EC2

Il sistema è stato portato su un'istanza AWS EC2 di tipo `t3.micro`, su cui eseguiamo un ambiente Ubuntu, dove il progetto è stato containerizzato ed eseguito tramite Docker Compose.

I tre nodi sono processati separatamente ed eseguiti in container sulla stessa macchina virtuale. Questa configurazione consente di sperimentare la disponibilità del singolo nodo del registry.

XI. TESTING

A. Test Service Registry completo

```
./scripts/test-service-registry.sh
```

Questo test verifica il comportamento dell'applicazione in un ambiente composto da vari container comunicanti fra loro.

Il test di integrazione simula il ciclo operativo di un cluster di Service Registry in ambiente Docker Compose. Eseguendo registrazioni, aggiornamenti e cancellazioni, controlla la propagazione dello stato, introduce i conflitti riproducibili mediante isolamento dei nodi e verifica il recupero dopo l'indisponibilità di un peer. Controlla inoltre la conservazione dei tombstone e dello stato completo dopo la ricreazione dei container con i volumi mantenuti.

1) registrazione e propagazione dell'istanza.

Il test registra l'istanza `payment-1` del servizio `payment` su `registry-1` tramite l'API `PUT/services/payment/instances/payment-1`.

2) interrogazione degli altri registry.

Tramite una `GET /services/payment` su `registry-2` e `registry-3`, viene verificato

se la creazione del servizio si è propagata agli altri nodi. Nel sistema, il gossip invia ai peer lo snapshot del registry in seguito a una modifica locale.

3) cancellazione mentre un nodo è offline.

Il `registry-3` viene fermato mentre il `registry-2` cancella l'istanza tramite una `DELETE /services/payment/instances/payment-1`. Verifichiamo che il `registry-1` veda la cancellazione. In questo modo possiamo essere certi che la cancellazione si propaghi anche quando un nodo non è attivo. La cancellazione viene rappresentata internamente da un record marcato con lo stato `deleted`. La discovery non mostra questo tipo di record, ma il registry lo conserva per comunicare agli altri nodi che l'istanza è stata eliminata.

4) recupero del nodo offline.

Il test riavvia `registry-3` e interroga il suo endpoint `GET /internal/state` per verificare la presenza del tombstone. Il nodo recupera lo stato attraverso il `Synchronizer`, che contatta i peer anche all'avvio.

5) preparazione dello stato comune.

Il test registra l'istanza `shared` del servizio `conflict`. E attende che gli stati completi dei tre nodi coincidano. Questa condizione permette di riprodurre scritture concorrenti a parità di versione.

6) verifica il conflitto aggiornamento/aggiornamento.

Per creare un conflitto riproducibile, il test ricrea i tre container, conservando i volumi ma impostando la lista dei peer vuota. Le API restano disponibili, mentre la replica è temporaneamente disabilitata. Vengono eseguiti in parallelo due aggiornamenti della stessa istanza, rispettivamente su `registry-1` e su `registry-2`. Il test verifica che entrambe le richieste riescano e che i record risultanti abbiano la stessa versione e origini diverse. Ripristinata la comunicazione, verifica che gli stati completi dei tre nodi coincidano con quello atteso: a parità di versione prevale l'aggiornamento di `registry-2`, la cui origine è lessicograficamente maggiore.

7) verifica conflitto aggiornamento/cancellazione.

Isoliamo nuovamente tutti e tre i nodi disabilitando la replica e conservando i volumi. Eseguiamo quindi in parallelo un aggiornamento su `registry-1` e una cancellazione della stessa istanza su `registry-2`. Verifichiamo che i record abbiano la stessa versione e origini diverse. In questo scenario, prevale la cancellazione in quanto `registry-2` ha origine lessicograficamente maggiore e non perché la cancellazione abbia priorità sugli aggiornamenti. Ripristinata la configurazione dei peer, il test verifica l'uguaglianza degli stati e l'assenza delle istanze nella discovery di tutti i nodi.

8) recupero di numerose modifiche.

Il test arresta nuovamente il `registry-3`. Viene avviato, un nuovo registry il `registry-4`. Esso è un nodo aggiuntivo con identità e volumi propri. Conosce come peer solo `registry-1` e `registry-2`.

Il registry-4:

- crea 100 istanze del servizio recovery-load.
- aggiorna l'istanza load-1, impostando un nuovo indirizzo e una nuova porta di ascolto.
- cancella l'ultima istanza, cioè la load-100.

Lo stato atteso è quindi di 99 istanze attive più il tombstone dell'istanza cancellata. Dopo la propagazione ai nodi disponibili e un ulteriore periodo di inattività, il registry-3 viene riavviato: deve recuperare, dai peer conosciuti, anche gli aggiornamenti originati da registry-4 che non compare nella sua lista dei peer noti.

Il numero di istanze aggiunte dal registry-4 può essere modificato tramite il parametro INSTANCE_COUNT.

9) misurazione e persistenza.

Il tempo di recupero è misurato all'avvio del comando che fa ripartire registry-3 fino al completamento dei primi controlli sullo stato recuperato. Comprende l'avvio del processo, la sincronizzazione e le verifiche HTTP. Successivamente viene verificata l'uguaglianza degli stati completi dei quattro nodi, confrontando tutti i campi inclusi versioni, origini, timestamp e tombstone. Per verificare la persistenza, il test conserva lo stato atteso, rimuove i container mantenendo i volumi e avvia soltanto registry-3. Confronta lo stato caricato con quello atteso e verifica che l'istanza cancellata non ricompaia nella discovery. Arresta quindi registry-3 e ripete il controllo con il solo registry-4. Poiché i peer non sono disponibili il controllo verifica il recupero dal volume locale.

I controlli vengono ripetuti con pause di un secondo fino al limite dei tentativi previsto. Se una condizione non è soddisfatta il test termina con un errore. Una procedura di pulizia rimuove container e volumi dall'ambiente di test anche in caso di errore.

B. Test di convergenza

La suite tests tests/convergence misura quantitativamente il comportamento del sistema in due scenari:

- 1) convergenza via gossip;
- 2) recupero tramite anti-entropia.

1) *Convergenza gossip*: Il test registra una nuova istanza su registry-1 e misura il tempo fino alla sua osservazione su registry-2 e su registry-3. La misura comprende la richiesta iniziale e le verifiche HTTP dei due destinatari, interrogati in sequenza. Dopo ogni controllo fallito e il test attende 10ms prima di riprovare; il primo controllo è immediato, quindi sono possibili misure inferiori a 10 ms. Ogni destinatario ha un timeout di attesa configurato di 5s.

2) *Recupero con anti-entropia*: Il test sospende registry-3 mediante docker compose pause. Durante la sua non-disponibilità vengono eseguite le seguenti azioni:

- 1) registrazione nuova istanza su registry-1.
- 2) attesa che sia visibile su registry-2.

- 3) mantenimento della sospensione del terzo nodo per altri 2,5s.
- 4) avvio del cronometro e riattivazione del nodo.
- 5) attesa risposta al controllo di disponibilità e verifica che l'istanza compaia nella discovery.

Il passaggio di 2,5 secondi serve a rendere significativo l'esperimento: il timeout delle richieste è di 2s a cui aggiungiamo 500 ms di margine per far scadere il tentativo di gossip verso il nodo sospeso. Il tempo trascorso offline, non entra nella misurazione. Il timeout configurato per l'attesa dell'istanza recuperata è di 10s.

XII. RISULTATI DEI TEST

I test utilizzano 20 campioni per ciascuno scenario e calcolano:

- minimo;
- media;
- mediana;
- 95° percentile;
- massimo;

Il polling delle API viene effettuato ogni 10ms.

Scenario	min	media	mediana	95 percentile	max
Gossip	1.215708	3.642343	1.553396	14.508709	15.224791
Anti-Entropia	81.161792	96.918287	98.148062	102.569	104.465125

Valori in ms.
TABLE I
RISULTATI LOCALI

Scenario	min	media	mediana	95 percentile	max
Gossip	2.069767	9.81143	12.627894	13.249948	13.600388
Anti-Entropia	123.610847	132.132124	128.164133	160.415488	163.566476

Valori in ms.
TABLE II
RISULTATI EC2

Nei 20 campioni raccolti per ciascuno scenario, la latenza media osservata per la propagazione dell'istanza è pari a 3,64 ms in locale e 9,81 ms su EC2; per il recupero del nodo sospeso è pari rispettivamente a 96,92 ms e 132,13 ms. Si tratta di misure end-to-end: lo scenario di propagazione comprende la registrazione e le verifiche HTTP, mentre quello di recupero comprende la riattivazione del container, il controllo di disponibilità e l'osservazione dell'istanza recuperata. I risultati descrivono quindi il comportamento dei due scenari nella configurazione sperimentale adottata. Le differenze tra l'ambiente locale e quello EC2 non consentono, da sole, di identificare il contributo dei singoli fattori infrastrutturali.

XIII. CONCLUSIONI

L'implementazione utilizza peer configurati staticamente e scambia snapshot completi, con un costo di comunicazione crescente al crescere dello stato. I tombstone sono conservati senza una procedura di eliminazione definitiva. Non sono previsti heartbeat o TTL per rilevare automaticamente

l'indisponibilità delle istanze registrate. La persistenza periodica non garantisce il salvataggio immediato di ogni scrittura confermata. Il deployment su un unico host consente di testare guasti dei singoli processi, ma non la perdita dell'intera macchina.